Vermont Health Platform — Troubleshooting Guide

Last updated: 2026-03-19 **Stack:** Next.js 14 (Vercel) · Python FastAPI backend (Railway) · Supabase (auth + database) · Sanity CMS · Stripe payments

This guide is organized by what you *see*, not by what caused it. Each section describes the exact symptom, the most likely causes ranked by probability, and step-by-step fixes you can run yourself.

Table of Contents

1. Auth & Login Problems
2. Supabase Problems
3. AI Chat Problems
4. Content / Sanity Problems
5. Frontend Problems
6. Python Backend Problems
7. Stripe & Payments Problems
8. General Problems

---

1. Auth & Login Problems

1.1 "Failed to fetch" on the login page

What you see: The login form shows a network error, or the browser console shows `Failed to fetch` / `net::ERR_CONNECTION_REFUSED` when you try to sign in.

Most likely causes:

1. `NEXT_PUBLIC_SUPABASE_URL` or `NEXT_PUBLIC_SUPABASE_ANON_KEY` is missing or wrong in your environment variables.
2. The Supabase project is paused (free plan pauses after 1 week of inactivity) — see Section 2.1 for the full fix.
3. The browser is blocked from reaching Supabase by the Content Security Policy (CSP).

Fix — Step by step:

Step 1: Check the environment variables are set

Local dev: Open `frontend/.env.local` and confirm these two lines are present and not blank:

```
NEXT_PUBLIC_SUPABASE_URL=https://yourref.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGci...
```

Production (Vercel):

1. Go to vercel.com → your project → **Settings** → **Environment Variables**
2. Confirm `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY` are listed under **Production**
3. If you just added or changed them, you must **redeploy**: click **Deployments** → the latest → **Redeploy**

Step 2: Confirm the Supabase project is running

Go to supabase.com/dashboard. If the project shows a **"Resume"** button, it is paused. Click Resume and wait 30–60 seconds.

Step 3: Verify the URL value itself

Copy the value of `NEXT_PUBLIC_SUPABASE_URL` and open `<that-url>/rest/v1/` in your browser. You should get a JSON response, not a timeout or 404. If you get an error, the project ref in the URL is wrong — go back to Supabase Dashboard → **Settings** → **API** and copy the URL again.

How to prevent it: Set up a Supabase Pro plan ($25/month) to avoid automatic pausing. Alternatively, visit the Supabase dashboard at least once a week to keep the free project active.

1.2 Stuck on "Signing in..."

What you see: You click the sign-in button, the button shows a spinner and says "Signing in...", and it never resolves — no error message, no redirect.

Most likely causes:

1. Supabase project is paused (auth calls time out silently).
2. A browser extension (ad blocker, privacy badger) is blocking requests to `*.supabase.co`.
3. `NEXT_PUBLIC_SUPABASE_ANON_KEY` is corrupted (extra space, truncated during copy-paste).

Fix — Step by step:

Step 1: Open browser DevTools Press **F12** (or right-click → Inspect) → click the **Network** tab → try signing in → look for any red (failed) requests to `*.supabase.co`.

Step 2: Check the Supabase dashboard Visit supabase.com/dashboard. Resume the project if it says "Paused" — see Section 2.1.

Step 3: Try a private / incognito window If sign-in works in a private window, a browser extension is the problem. Disable them one by one.

Step 4: Verify the anon key In Supabase Dashboard → **Settings** → **API**, copy the `anon` key again. It should start with `eyJ` and be about 200+ characters long. Paste it into your env file (no leading/trailing spaces) and restart the dev server:

```
cd frontend && npm run dev
```

1.3 Redirected to /upgrade after login

What you see: You log in successfully, but the page sends you to `/upgrade` (or `/upgrade?from=/dashboard`) instead of letting you in.

Most likely cause: Your Supabase account exists but has no role in the `user_roles` table, or only has the `free` role — which does not grant access to subscriber-only routes like `/dashboard`, `/chat`, or `/hti-dashboard`.

The middleware checks the `user_roles` table and compares against a role hierarchy: `free → subscriber → student → professional → advisory → admin`

Access to `/dashboard` requires `subscriber` or higher. If the row is missing or says `free`, you are redirected to `/upgrade`.

Fix — Step by step:

Step 1: Check your user's roles in Supabase

1. Go to supabase.com/dashboard → your project
2. Click **Table Editor** in the left sidebar → select the `user_roles` table
3. Search for your user ID (you can find your user ID in **Authentication** → **Users** → click your email)
4. Look at the `role` column for rows matching your user ID

Step 2: Manually grant a role (for your own account or test users) In the Supabase **SQL Editor** (left sidebar), run:

```
-- Replace the UUID and role with correct values
INSERT INTO user_roles (user_id, role)
VALUES ('your-user-uuid-here', 'subscriber')
ON CONFLICT (user_id, role) DO NOTHING;
```

Step 3: If this happened after a payment A payment was processed but the webhook that grants the role failed. See Section 7.2 for the full fix.

How to prevent it: After Stripe webhooks are tested, role grants happen automatically within seconds of checkout completion. Keep `STRIPE_WEBHOOK_SECRET` correct and the webhook endpoint registered in Stripe.

1.4 Can't access /dashboard after login

What you see: You are logged in, you can see your account, but going to `/dashboard` redirects you to `/login` or `/upgrade`.

Most likely causes:

- Redirected to `/login`: Your session cookie expired or was not set. The middleware found no authenticated user.
- Redirected to `/upgrade`: You are authenticated but lack the `subscriber` (or higher) role. See Section 1.3.

Fix — Redirect to /login:

Step 1: Sign out and sign back in Your session token may be stale. Sign out completely, clear your browser cookies for this domain (DevTools → Application tab → Cookies → Clear all for your domain), then sign in fresh.

Step 2: Check cookie settings The app uses Supabase SSR cookies. If your site is accessed over HTTP (not HTTPS) in production, cookies with the `Secure` flag will not be set. Make sure your production URL starts with `https://`.

Step 3: Confirm middleware is not caching a stale redirect If using Vercel Edge Caching, ensure the `/dashboard` route is not being cached. The middleware runs on every request and should not be cached.

1.5 Session expires too quickly

What you see: You are signed in, walk away for a few hours, come back, and are logged out — even though you did not click "Sign out."

Most likely cause: Supabase JWT access tokens expire after 1 hour by default. The app should automatically refresh them using the refresh token. If refresh fails, you get logged out.

Fix — Step by step:

Step 1: Check if refresh tokens are enabled In Supabase Dashboard → **Authentication** → **Settings** → scroll to **JWT expiry** and **Refresh token** settings. Confirm "Enable refresh tokens" is ON.

Step 2: Check for cookie domain mismatch If your app is on `app.yourdomain.com` but the cookie is set for `yourdomain.com`, the SSR client may not see the refresh token on the server side. Confirm `NEXT_PUBLIC_SUPABASE_URL` matches the project used in production (not a dev project).

Step 3: Extend the token lifetime (temporary workaround) In Supabase Dashboard → **Authentication** → **Settings** → **JWT expiry**, increase the expiry to 3600 seconds (1 hour) or higher. Note: this is a security trade-off; longer expiry = longer window if a token is stolen.

How to prevent it: Ensure your Next.js app is calling `supabase.auth.getSession()` or `supabase.auth.getUser()` on protected pages, which triggers an automatic token refresh. The middleware already does this on every request.

1.6 Google OAuth not working

What you see: You click "Sign in with Google," Google asks you to authorize, and then you are sent back to the app with an error, or you are redirected to the wrong URL, or nothing happens.

Most likely causes:

1. The **Redirect URL** in Google Cloud Console does not match what Supabase sends.
2. Google OAuth provider is not enabled in Supabase.
3. The Google Client ID / Secret in Supabase is wrong or outdated.

Fix — Step by step:

Step 1: Enable Google in Supabase

1. Supabase Dashboard → **Authentication** → **Providers** → **Google**
2. Toggle **Enable** to ON
3. Enter your **Google Client ID** and **Google Client Secret** (from Google Cloud Console)

Step 2: Set the Authorized Redirect URI in Google Cloud Console

1. Go to console.cloud.google.com → your project → **APIs & Services** → **Credentials**
2. Click your OAuth 2.0 Client ID
3. Under **Authorized redirect URIs**, add:
   - `https://yourref.supabase.co/auth/v1/callback` (replace `yourref` with your Supabase project ref)
   - If testing locally: `http://localhost:3000/auth/callback`
4. Click **Save**

Step 3: Set the Site URL in Supabase

1. Supabase Dashboard → **Authentication** → **URL Configuration**
2. **Site URL**: set to your production domain, e.g. `https://yourapp.vercel.app`
3. **Additional Redirect URLs**: add `http://localhost:3000` for local testing
4. Save

Step 4: Clear browser cookies and try again Google OAuth flows use redirects that can get confused by stale session cookies. Open a private window to test.

---

2. Supabase Problems

2.1 Supabase Project is Paused (Most Common Issue)

What you see: Everything was working, then suddenly:

- Login shows "Failed to fetch"
- The AI chat does not authenticate
- Database queries fail silently
- The API health check shows errors

Why this happens: Supabase free-tier projects are automatically paused after **7 days of inactivity**. All database and auth API calls fail while paused. This is the single most common cause of the app going down.

Fix — Step by step:

Step 1: Go to the Supabase dashboard Open supabase.com/dashboard in your browser. Log in with your Supabase account.

Step 2: Find the paused project Your project will show a yellow/orange badge that says **"Paused"** or a **"Resume"** button.

Step 3: Resume the project Click the **"Resume project"** button. A dialog will appear asking you to confirm. Click **"Resume"**.

Step 4: Wait for restart The project takes **30–90 seconds** to fully restart. The dashboard will show a progress indicator. Do not navigate away.

Step 5: Verify it is working After the progress bar completes, go to **Table Editor** and click any table. If you can see data, the project is live. Then reload your app and try logging in.

If the Resume button does not appear:

- Check that you are on the correct Supabase account (you may have multiple Google accounts)
- Try refreshing the dashboard page
- If the project is deleted (not just paused), you will need to restore from a backup — contact Supabase support

How to prevent this permanently:

Option A (recommended): Upgrade to **Supabase Pro** ($25/month). Pro projects never pause.

1. Supabase Dashboard → your project → **Settings** → **Billing** → **Upgrade to Pro**

Option B (free workaround): Set up a free uptime monitor (UptimeRobot, BetterStack free tier) that pings your app's URL every 5 minutes. This counts as activity and keeps the Supabase project from pausing.

1. Go to uptimerobot.com → create a free account
2. Add a new monitor: HTTP(S) type, URL = `https://yourapp.vercel.app/api/health`, interval = 5 minutes
3. That's it — the periodic pings will keep Supabase active

Option C: Manually visit the Supabase dashboard or make a database query at least once a week.

2.2 "relation does not exist" SQL errors

What you see: Server logs (Vercel function logs, or your terminal running the dev server) show errors like:

```
relation "user_roles" does not exist
relation "subscriptions" does not exist
relation "stripe_events" does not exist
```

Most likely causes:

1. The database schema migrations have not been run on this Supabase project (e.g., you created a new project and restored data but not the schema).
2. The project was deleted and recreated, losing the schema.
3. The query is running against the wrong Supabase project (wrong URL in env vars).

Fix — Step by step:

Step 1: Verify you are connected to the right project Check `NEXT_PUBLIC_SUPABASE_URL` in your env. The project ref in the URL must match the project you see in the Supabase dashboard.

Step 2: Check which tables exist In Supabase Dashboard → **Table Editor** → look at the left sidebar list of tables. You should see: `user_roles`, `subscriptions`, `stripe_events`, `stripe_customers`.

Step 3: Run the missing migrations Go to Supabase Dashboard → **SQL Editor** → **New query**, then run the schema setup scripts. Check the `supabase/` folder in the project for `.sql` files:

```
ls /Users/baba/Vermont-Health-Platform/supabase/
```

Open each `.sql` file and paste it into the Supabase SQL Editor, then click **Run**.

Step 4: If you do not have migration files The minimum required tables for the app are:

```sql
-- user_roles
CREATE TABLE IF NOT EXISTS user_roles (
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  role TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (user_id, role)
);

-- subscriptions
CREATE TABLE IF NOT EXISTS subscriptions (
  user_id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  stripe_customer_id TEXT,
  stripe_subscription_id TEXT,
  plan TEXT DEFAULT 'free',
  status TEXT DEFAULT 'active',
  current_period_start TIMESTAMPTZ,
  current_period_end TIMESTAMPTZ,
  cancel_at_period_end BOOLEAN DEFAULT false,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- stripe_customers
CREATE TABLE IF NOT EXISTS stripe_customers (
  user_id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  stripe_customer_id TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- stripe_events (idempotency log)
CREATE TABLE IF NOT EXISTS stripe_events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id TEXT UNIQUE NOT NULL,
  event_type TEXT NOT NULL,
  payload JSONB,
  created_at TIMESTAMPTZ DEFAULT now()
);
```

Run this SQL in Supabase SQL Editor → click **Run**.

2.3 RLS policy blocking a query

What you see: A database read or write silently returns empty results or a `403` error when it should return data. In server logs you may see:

```
new row violates row-level security policy for table "user_roles"
```

Or a query returns `[]` (empty array) when you know data exists in the table.

Most likely causes:

1. Row Level Security (RLS) is enabled on the table but the policy does not allow the current user (or the service role) to read/write it.
2. The app is using the **anon key** for an operation that requires the **service role key**.
3. A new table was created without setting up RLS policies.

How to diagnose:

In Supabase Dashboard → **Authentication** → **Policies** (or **Table Editor** → click a table → **RLS**), look for the table in question. If "RLS Enabled" is ON but there are no policies listed, **all queries to that table will be blocked for non-admin roles**.

Fix — Step by step:

Step 1: Check if the operation uses the service role Webhook handlers (`/api/stripe/webhook`) and server-side admin operations use `dbAdmin` (the service role client). The service role bypasses RLS entirely — if these operations are failing, the issue is likely a wrong service role key, not RLS (see Section 2.4).

Step 2: For client-side queries (anon key / user session) The app's middleware queries `user_roles` using the user's session (anon key). This requires a policy like:

```
-- Allow users to read their own roles
CREATE POLICY "Users can read own roles"
ON user_roles FOR SELECT
USING (auth.uid() = user_id);
```

Go to Supabase Dashboard → **SQL Editor** → run the appropriate policy.

Step 3: Temporarily disable RLS to test (NOT for production) If you just need to confirm that RLS is the problem, you can temporarily disable it:

```
ALTER TABLE user_roles DISABLE ROW LEVEL SECURITY;
```

If your query now works, RLS is the cause. Re-enable it and add the correct policy:

```
ALTER TABLE user_roles ENABLE ROW LEVEL SECURITY;
```

How to prevent it: Whenever you create a new table in Supabase, immediately add RLS policies for it before using it in the app. The Supabase dashboard will warn you with a yellow shield icon if a table has RLS enabled but no policies.

2.4 Service role key rejected

What you see: Server-side operations (webhook processing, admin user lookups) fail with:

```
Invalid API key
JWT expired
```

Or Stripe webhook handler returns a 500 error even though the signature is valid.

Most likely causes:

1. `SUPABASE_SERVICE_ROLE_KEY` is missing from the environment.
2. The key was rotated in Supabase but the environment variable was not updated.
3. The key in the env file has a copy-paste artifact (space, newline, truncation).

Fix — Step by step:

Step 1: Get the correct service role key

1. Supabase Dashboard → **Settings** → **API**
2. Find the **service_role** key under "Project API keys" — it is labeled "secret" and hidden by default
3. Click the eye icon to reveal it, then copy the entire key

Step 2: Update the environment variable

Local (`backend/.env`):

```
SUPABASE_SERVICE_ROLE_KEY=eyJhbGci...the_full_key
```

Production (Vercel):

1. Vercel Dashboard → your project → **Settings** → **Environment Variables**
2. Edit `SUPABASE_SERVICE_ROLE_KEY` → paste the fresh key → Save
3. Go to **Deployments** → Redeploy the latest deployment

Step 3: Verify the key length A valid Supabase service role JWT is typically 200+ characters starting with `eyJ`. If yours is shorter, it is truncated.

Important: Never expose `SUPABASE_SERVICE_ROLE_KEY` to the browser. It must only appear in server-side env vars (never prefixed with `NEXT_PUBLIC_`).

---

3. AI Chat Problems

3.1 "AI Offline" indicator

What you see: The chat page shows an "AI Offline" badge or indicator. Clicking the chat input shows a message like "AI is currently unavailable."

How the indicator works: The frontend polls `/api/health` (a Next.js route), which itself calls `http://localhost:8000/health` (or your `PYTHON_BACKEND_URL`). If the Python backend does not respond, `/api/health` returns `{ ok: false, indexReady: false }` and the UI shows "AI Offline."

Most likely causes:

1. The Python backend process is not running (most common in local dev).
2. `PYTHON_BACKEND_URL` in `frontend/.env.local` (or Vercel) is wrong or pointing to a stopped server.
3. The backend crashed on startup (missing env var, missing Python package).

Fix — Step by step (local development):

Step 1: Start the Python backend Open a terminal (separate from the one running `npm run dev`) and run:

```
cd /Users/baba/Vermont-Health-Platform/backend
uvicorn main:app --reload --port 8000
```

Watch for errors in the output. If it starts cleanly, you will see:

```
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Step 2: Test the backend directly Open your browser and go to `http://localhost:8000/health`. You should see JSON like `{"status":"ok","index_ready":true}`. If you see a connection error, the backend is not running.

Step 3: Check for startup errors If the backend crashes immediately on start, the terminal will show a Python traceback. Common causes:

- `GOOGLE_API_KEY is required` → add `GOOGLE_API_KEY=...` to `backend/.env`
- `ModuleNotFoundError` → run `pip install -r backend/requirements.txt`
- Port 8000 already in use → run `lsof -i :8000` and kill the conflicting process

Fix — Step by step (production on Railway):

1. Go to your Railway project → click the backend service
2. Check **Deployments** — is the latest deployment green (running) or red (crashed)?

3. Click the deployment → **View Logs** — look for the Python traceback that caused the crash
4. The most common production crash: a missing environment variable. Go to **Variables** tab and confirm all variables from `backend/.env.example` are set

---

3.2 Chat returns an error message

What you see: You type a message and hit send, but instead of an AI response you see an error message in the chat window, such as:

- "The AI backend returned an error. Is the Python server running?"
- "Cannot reach the AI backend."
- "Your plan does not include AI Analyst access. Upgrade to Subscriber or higher."

Most likely causes and fixes:

"Cannot reach the AI backend" → Backend is not running. See Section 3.1.

"Your plan does not include AI Analyst access" → Your Supabase user account has the `free` role, not `subscriber` or higher. The Python backend validates your JWT and rejects free-tier users. Fix: grant yourself a paid role — see Section 1.3.

"The AI backend returned an error" → The backend is running but returned a 5xx error. Check the Python backend terminal for a Python traceback. Common causes:

- Google API key rate limit exceeded — see Section 6.4
- Supabase pgvector connection failing — check `SUPABASE_DB_URL` in `backend/.env`
- Malformed conversation history — try clearing your chat and starting fresh

---

3.3 Chat is extremely slow

What you see: You send a message and wait 30–120 seconds before anything appears. Or the chat times out entirely.

Most likely causes:

1. The vector index is being rebuilt from scratch (first startup or after clearing `backend/storage/`). This takes 2–5 minutes.
2. The Groq API is responding slowly (upstream latency — not something you can control directly).
3. The Supabase pgvector query is slow because embeddings are not indexed.

Fix — Step by step:

Step 1: Check if it is an index rebuild Look at the Python backend terminal. If you see lines like:

```
INFO: Ingesting PDFs from backend/data/...
INFO: Fetching Sanity content...
INFO: Building vector index...
```

The index is being rebuilt. Wait for it to complete (5–10 minutes on first run). Subsequent starts will be fast (~2 seconds) because the index is cached.

Step 2: Check the health endpoint Go to `http://localhost:8000/health` (local) or your backend's `/health` URL. Look at the `index_ready` field:

- `"index_ready": false` → index is still building; chat will be slow or unavailable
- `"index_ready": true` → index is ready; slowness is upstream latency

Step 3: If it is consistently slow (index is ready) The Groq LLM may be experiencing high latency. Try:

- Waiting a few minutes and trying again
- Checking Groq's status page

Step 4: Add a pgvector index (production) In Supabase SQL Editor, if not already present:

```
CREATE INDEX IF NOT EXISTS rag_documents_embedding_idx
ON rag_documents USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);
```

This dramatically speeds up vector similarity searches.

3.4 Responses are irrelevant or hallucinating

What you see: The AI chat gives answers that seem made up, unrelated to Vermont Health Platform content, or confidently wrong about specific program details.

Most likely causes:

1. The vector index was built with no or few source documents in `backend/data/`.
2. Sanity CMS content is empty — the AI has no real content to ground responses in.
3. The index is stale and does not include recently added content.

Fix — Step by step:

Step 1: Verify source documents exist

```
ls /Users/baba/Vermont-Health-Platform/backend/data/
```

You should see PDF files. If this directory is empty, the AI has no source material and will hallucinate. Add PDF files (reports, policy documents, etc.) to `backend/data/`.

Step 2: Trigger a re-index After adding new PDFs or Sanity content, force the backend to rebuild its index:

```
curl -X POST http://localhost:8000/api/ingest \
  -H "Content-Type: application/json" \
  -d '{"secret": "your_ingest_secret_here"}'
```

Replace `your_ingest_secret_here` with the value of `INGEST_SECRET` from `backend/.env`.

Step 3: Add content in Sanity Log into your Sanity Studio (`/studio` route on your app, or sanity.io/manage). Add posts, policy analyses, case studies, and other content. After publishing, re-trigger the ingest (Step 2).

3.5 Rate limit hit (429 error)

What you see: The chat responds with:

```
"Too many requests. Please wait a moment before sending another message."
```

Or the browser console shows a `429` response.

What this means: The app enforces a limit of **30 chat messages per user per 60 seconds**. Hitting this means you sent too many messages too quickly.

Fix: Wait 60 seconds, then try again. The rate limit window resets automatically.

If legitimate users are hitting this regularly: The rate limit is defined in `frontend/lib/rate-limit.ts`. To increase the limit, edit the values in `frontend/app/api/chat/route.ts`:

```
const limiter = await rateLimit(`chat:${session.user.id}`, {
  limit: 30,    // increase this number
  window: 60_000,
});
```

Then redeploy. Note: higher limits increase your Google API costs.

---

4. Content / Sanity Problems

4.1 Page shows no content

What you see: A page that should display articles, modules, or other CMS content is blank, shows a loading skeleton forever, or shows "No content found."

Most likely causes:

1. Sanity content has not been published (it exists as a draft).
2. `NEXT_PUBLIC_SANITY_PROJECT_ID` or `NEXT_PUBLIC_SANITY_DATASET` env vars are wrong.
3. The Sanity API token (`SANITY_API_TOKEN`) is missing or invalid for pages that require a token.
4. The page is fetching from the `production` dataset but content was added to a different dataset.

Fix — Step by step:

Step 1: Check if content is published in Sanity

1. Go to your Sanity Studio (`https://yourapp.vercel.app/studio` or run `cd frontend && npm run dev` locally and visit `localhost:3000/studio`)
2. Navigate to the content type (e.g., Post, Policy Analysis)
3. Open the document — if you see a **green "Published"** badge, it is live. If it shows **"Draft"**, click **Publish**

Step 2: Verify Sanity env vars In your `frontend/.env.local` (local) or Vercel environment variables (production), confirm:

```
NEXT_PUBLIC_SANITY_PROJECT_ID=your_actual_project_id
NEXT_PUBLIC_SANITY_DATASET=production
NEXT_PUBLIC_SANITY_API_VERSION=2023-10-01
```

The project ID is in sanity.io/manage → your project → **API** tab.

Step 3: Test the Sanity API directly Open this URL in your browser (replace `YOURPROJECTID`):

```
https://YOURPROJECTID.api.sanity.io/v2023-10-01/data/query/production?query=*
[_type=="post"][0..2]
```

If you see JSON with `result: []`, there is no published content. If you see an error, the project ID or dataset name is wrong.

4.2 Images not loading

What you see: Images from Sanity (article thumbnails, author photos) show broken image icons or empty boxes. The browser console shows `ERR_BLOCKED_BY_CSP` or a 404.

Most likely causes:

1. `ERR_BLOCKED_BY_CSP` → The Content Security Policy blocks the image source domain.
2. 404 → The image URL is correct but the Sanity CDN asset no longer exists (asset was deleted from Sanity).

3. The image src URL references the wrong Sanity project ID.

Fix — Step by step:

For CSP errors (`ERR_BLOCKED_BY_CSP`):

Sanity images are served from `cdn.sanity.io`, which is already allowed in `frontend/next.config.ts`:

```
"img-src 'self' blob: data: https://cdn.sanity.io https://img.youtube.com"
```

If you see a CSP error for an image from a different domain, you need to add that domain to the `img-src` line in `next.config.ts` and redeploy.

For 404 image errors:

1. Open Sanity Studio → find the content item with the broken image
2. Remove the image and re-upload it
3. Click **Publish** to save

4.3 Glossary / definitions page is empty

What you see: The glossary or definitions page shows no entries, or shows "No definitions found."

Most likely cause: No `definition` documents have been published in Sanity.

Fix:

1. Open Sanity Studio → in the left sidebar, look for **Definitions** or **Glossary**
2. Create and publish at least one definition document
3. The page will refresh (Next.js revalidates on a schedule; in dev you may need to restart the dev server)

4.4 Search returns no results

What you see: Using the search feature returns empty results for queries that should match content.

Most likely causes:

1. Search is not yet fully implemented (it is on the roadmap after content depth).
2. The search index has not been built.
3. Sanity content has no published documents to search.

Fix: Check `frontend/app/api/search/route.ts` to understand the current implementation. If search is returning empty, confirm that content exists in Sanity (Step 1 from Section 4.1), then check the search API route for configuration requirements.

---

5. Frontend Problems

5.1 Build fails

What you see: Running `npm run build` in the `frontend/` directory exits with errors, and your Vercel deployment shows a "Build failed" status.

Most likely causes:

1. A TypeScript type error was introduced.
2. A new import references a module that does not exist.
3. An environment variable used at build time is missing (e.g., `NEXT_PUBLIC_SANITY_PROJECT_ID`).

Fix — Step by step:

Step 1: Read the exact error The build output will show the exact file and line number. In Vercel, click on the failed deployment → **Build Logs** → scroll to the first red error line.

Step 2: Run the build locally

```
cd /Users/baba/Vermont-Health-Platform/frontend
npm run build
```

This replicates the Vercel build locally so you can see errors in your terminal.

Step 3: Run TypeScript check separately (faster)

```
cd /Users/baba/Vermont-Health-Platform/frontend
npx tsc --noEmit
```

This catches all type errors without running the full build.

Step 4: Check for missing env vars at build time If the error mentions `undefined` where a URL or key is expected, a build-time env var is missing. In Vercel → **Settings** → **Environment Variables**, confirm all `NEXT_PUBLIC_*` vars are set and marked for the **Build** environment (not just Runtime).

Step 5: Clear the Next.js cache and retry

```
cd /Users/baba/Vermont-Health-Platform/frontend
rm -rf .next
npm run build
```

5.2 "Failed to fetch" in browser console (CSP issues)

What you see: The app loads but certain features do not work — buttons do nothing, chat does not connect, maps do not load. The browser console (F12 → Console) shows:

```
Refused to connect to 'https://...' because it violates the following Content
Security Policy directive: "connect-src ..."
```

What this means: The Content Security Policy (CSP) in `frontend/next.config.ts` is blocking an outgoing network request to a domain that is not on the allowlist.

Most likely causes:

1. A new third-party API or service was added to the app but its domain was not added to the CSP.
2. The Supabase project URL changed (e.g., you moved to a different project with a different region).
3. A new external font, script, or image source was added.

Fix — Step by step:

Step 1: Identify the blocked domain from the console error The error message will include the exact URL that was blocked, e.g.:

```
Refused to connect to 'https://api.newservice.io/...'
```

Step 2: Add the domain to the CSP in `next.config.ts` Open `/Users/baba/Vermont-Health-Platform/frontend/next.config.ts`. Find the relevant directive:

- API calls → `connect-src`
- Images → `img-src`
- Scripts → `script-src`
- Iframes (embeds) → `frame-src`
- Fonts → `font-src`

Add the domain to the appropriate directive. Example — adding a new API domain to `connect-src`:

```
"connect-src 'self' https://*.supabase.co ... https://api.newservice.io",
```

Step 3: Restart the dev server or redeploy CSP changes require a server restart (local) or a new Vercel deployment (production).

5.3 Page shows 404 unexpectedly

What you see: A page that should exist returns a 404 error ("This page could not be found").

Most likely causes:

1. The route file was accidentally deleted or moved.
2. A dynamic route (e.g., `/states/[state]`) received a slug that has no matching data.
3. The Vercel deployment is serving a cached old version.

Fix — Step by step:

Step 1: Check if the page file exists For a URL like `/vermont-act-167`, the file should be at:

```
frontend/app/vermont-act-167/page.tsx
```

Check:

```
ls /Users/baba/Vermont-Health-Platform/frontend/app/vermont-act-167/
```

Step 2: For dynamic routes, check the data For a URL like `/states/north_carolina`, the state ID `north_carolina` must exist in `frontend/lib/data/state-initiatives-data.ts`. If the state ID is missing or spelled differently, the page returns 404. The state ID format is `name.toLowerCase().replace(/\s+/g, "_")`.

Step 3: Force a fresh Vercel deployment In Vercel Dashboard → your project → **Deployments** → click **Redeploy** on the latest deployment. This clears the edge cache.

5.4 Changes to next.config.ts not taking effect

What you see: You edited `next.config.ts` (e.g., added a new CSP domain or a new env var), but the behavior has not changed after saving.

Fix:

Changes to `next.config.ts` require a **full server restart**. Pressing Ctrl+S does not apply these changes to a running `npm run dev` process.

Local:

1. Press `Ctrl+C` in the terminal running `npm run dev`
2. Run `npm run dev` again

Production (Vercel): `next.config.ts` changes are applied at build time. You must trigger a new deployment:

1. Commit and push your change to the main branch (if Vercel is connected to GitHub)
2. Or: Vercel Dashboard → **Deployments** → **Redeploy**

---

6. Python Backend Problems

6.1 Backend won't start

What you see: Running `uvicorn main:app --reload --port 8000` in the `backend/` directory immediately exits with an error, or crashes before printing "Uvicorn running on..."

Common error messages and fixes:

`ValueError: GOOGLE_API_KEY is required in backend/.env` The `.env` file is missing or does not contain this key.

```
# Check the file exists
ls /Users/baba/Vermont-Health-Platform/backend/.env
# If missing, copy the example
cp /Users/baba/Vermont-Health-Platform/backend/.env.example
/Users/baba/Vermont-Health-Platform/backend/.env
# Then open it and fill in your GOOGLE_API_KEY
```

`ModuleNotFoundError: No module named 'fastapi'` (or any other module) Python dependencies are not installed.

```
cd /Users/baba/Vermont-Health-Platform/backend
pip install -r requirements.txt
```

If `pip` is not found, use `pip3`. If you use a virtual environment, activate it first:

```
source venv/bin/activate  # or: source .venv/bin/activate
pip install -r requirements.txt
```

`OSError: [Errno 48] Address already in use: ('0.0.0.0', 8000)` Something is already running on port 8000.

```
# Find what is using port 8000
lsof -i :8000
# Kill it (replace 12345 with the PID from the output above)
kill -9 12345
# Now start the backend
uvicorn main:app --reload --port 8000
```

`ImportError: No module named 'llama_index.llms.groq'` The LlamaIndex Groq package is missing.

```
pip install llama-index-llms-groq
```

6.2 Index rebuild taking forever

What you see: The backend starts but logs show it has been ingesting and building the index for more than 10 minutes:

```
INFO: Ingesting PDFs from backend/data/...
INFO: Fetching Sanity content...
INFO: Building vector index...
```

The `index_ready` field in `/health` stays `false`.

Most likely causes:

1. A large number of PDF files in `backend/data/` are being processed.
2. Sanity GROQ API is slow or timing out during content fetch.
3. Supabase pgvector upsert is slow because the table has no index.
4. The Gemini Embeddings API is being rate-limited, causing retries.

Fix — Step by step:

Step 1: Monitor progress in the terminal The backend logs each major step. Watch for which step is stuck. If it says "Fetching Sanity content" for more than 2 minutes, there may be a Sanity API issue.

Step 2: Reduce the amount of data being indexed (temporary) If you have many large PDFs in `backend/data/`, temporarily move some out to a backup folder to speed up the first run:

```
mkdir /Users/baba/Vermont-Health-Platform/backend/data_backup
mv /Users/baba/Vermont-Health-Platform/backend/data/large_file.pdf
/Users/baba/Vermont-Health-Platform/backend/data_backup/
```

Add them back once the initial index is built.

Step 3: Check for Sanity API errors If the backend log shows Sanity fetch errors or timeouts, check your `SANITY_API_TOKEN` and `SANITY_PROJECT_ID` in `backend/.env`.

Step 4: Check the Gemini embedding quota (if embeddings are failing) See Section 6.4. Embedding quota exhaustion causes retries that make indexing take forever.

Normal build time expectations:

- First run with < 5 PDFs and moderate Sanity content: **2–5 minutes**
- First run with 20+ PDFs: **10–20 minutes**
- Subsequent starts (index cached in `backend/storage/` or Supabase): **2–5 seconds**

6.3 Out of memory during indexing

What you see: The backend process is killed during the index build, with an error like:

```
Killed
MemoryError
```

Or the Railway deployment shows the process restarting repeatedly.

Most likely causes:

1. Too many large PDFs being processed at once.
2. The Railway / server instance has insufficient RAM (the free tier has 512 MB).

Fix — Step by step:

Step 1: Reduce PDF size and count PDFs over 50 MB are unusual for policy documents. If you have large PDFs:

- Split them into smaller files (use Preview on Mac: File → Export as PDF, select page range)

- Remove image-heavy PDFs that are not text-searchable

Step 2: Upgrade the Railway instance In Railway → your backend service → **Settings** → increase the RAM allocation. The backend typically needs 512 MB minimum; 1 GB is recommended for 10+ PDFs.

Step 3: Increase Python memory efficiency The backend processes documents in batches. This is already handled by LlamaIndex, but if memory is still an issue, process PDFs one at a time by keeping only a few files in `backend/data/` and re-ingesting incrementally.

6.4 API quota exceeded

There are two separate quota pools — Groq (chat) and Gemini (embeddings). Errors look different for each.

Groq quota exceeded (chat):

```
groq.RateLimitError: Rate limit reached for llama-3.3-70b-versatile
```

Chat responses will fail with a streaming error. The index is unaffected.

Fix:

- Per-minute quota: wait 60 seconds and retry — resets automatically.
- Per-day quota: wait until midnight UTC or upgrade your Groq plan at https://console.groq.com.

Gemini embedding quota exceeded (index build only):

```
RuntimeError: Gemini embedding quota exceeded
```

Only happens during `/api/ingest` or first-run index build. Chat is unaffected if the index was previously built.

Fix — Step by step:

Step 1: Wait for the quota to reset Gemini free tier resets daily. Wait until the next day and re-trigger with:

```
curl -X POST http://localhost:8000/api/ingest
```

Step 2: Check your quota usage Go to Google AI Studio → **Manage API keys** → view quota usage.

Step 3: Enable billing on your Google Cloud project Creates a billing-enabled project, generates a new API key, and updates `GOOGLE_API_KEY` in `backend/.env`. Usage-based billing applies but limits are much higher than the free tier.

1. Update `GOOGLE_API_KEY` in `backend/.env` with the new key
2. Restart the backend

---

7. Stripe & Payments Problems

7.1 Webhook signature verification failed

What you see: In Vercel function logs (or your Next.js server logs), you see:

```
Webhook signature verification failed: Error: No signatures found matching the
expected signature for payload.
```

Stripe Dashboard → **Webhooks** → your endpoint shows failed deliveries with `400` status.

Why this happens: Stripe sends a `stripe-signature` header with every webhook. The app verifies it using `STRIPE_WEBHOOK_SECRET`. If the secret is wrong, the signature check fails and the webhook is rejected (no roles are granted, subscriptions are not updated).

Fix — Step by step:

Step 1: Get the correct webhook secret

1. Go to dashboard.stripe.com → **Developers** → **Webhooks**
2. Click on your webhook endpoint (the URL should be `https://yourapp.vercel.app/api/stripe/webhook`)
3. Under "Signing secret," click **Reveal** and copy the value — it starts with `whsec_`

Step 2: Update the environment variable

Vercel:

1. Vercel Dashboard → your project → **Settings** → **Environment Variables**
2. Edit `STRIPE_WEBHOOK_SECRET` → paste the `whsec_...` value → Save
3. Redeploy: **Deployments** → Redeploy

Step 3: Verify the webhook URL in Stripe matches your actual app URL In Stripe Dashboard → **Webhooks** → check that the endpoint URL is exactly `https://yourapp.vercel.app/api/stripe/webhook` (no trailing slash, correct domain).

Step 4: Check you are using the right Stripe mode If your app is in production but `STRIPE_WEBHOOK_SECRET` is from a **test mode** webhook, it will fail. Make sure you are using:

- Live mode keys (`sk_live_...`, `pk_live_...`, `whsec_...` from live webhooks) for production
- Test mode keys (`sk_test_...`, `pk_test_...`, `whsec_...` from test webhooks) for local development

Local webhook testing: When running locally, Stripe cannot reach `localhost:3000`. Use the Stripe CLI:

```
stripe listen --forward-to localhost:3000/api/stripe/webhook
```

The CLI will print a temporary webhook secret — use that as your local `STRIPE_WEBHOOK_SECRET`.

7.2 User paid but still on free plan

What you see: A user successfully completed a Stripe checkout (they have a receipt email), but their account still shows the free plan and they cannot access subscriber content.

What should happen: After successful checkout, Stripe fires a `checkout.session.completed` webhook → the app receives it → the `user_roles` table is updated to grant the `subscriber` role.

Most likely causes:

1. The webhook was not received (wrong URL, wrong secret — see Section 7.1).
2. The webhook was received but the handler crashed (Supabase error, missing `supabase_user_id` in session metadata).
3. The webhook secret was just fixed, but the past failed webhooks were never retried.

Fix — Step by step:

Step 1: Check Stripe for the failed webhook delivery

1. Stripe Dashboard → **Developers** → **Webhooks** → your endpoint
2. Click on recent deliveries — look for `checkout.session.completed` events with a red/failed status
3. Click on a failed delivery to see the response body (it will show the error from your app)

Step 2: Retry the webhook from Stripe On the failed webhook delivery detail page, click **Resend**. Stripe will replay the event. If your webhook endpoint is now working, this will grant the role automatically.

Step 3: Manually grant the role (immediate fix for the affected user) In Supabase SQL Editor:

```
-- Find the user by email first
SELECT id FROM auth.users WHERE email = 'user@example.com';

-- Then grant the role (replace UUID with the result above)
DELETE FROM user_roles WHERE user_id = 'user-uuid' AND role = 'free';
INSERT INTO user_roles (user_id, role)
VALUES ('user-uuid', 'subscriber')
ON CONFLICT (user_id, role) DO NOTHING;
```

Step 4: Also update the subscriptions table

```
INSERT INTO subscriptions (user_id, plan, status)
VALUES ('user-uuid', 'subscriber', 'active')
ON CONFLICT (user_id) DO UPDATE SET plan = 'subscriber', status = 'active';
```

How to prevent it: After fixing the webhook secret, click **Resend** on all recent failed webhook deliveries so no paid users are left on the free plan.

7.3 Checkout session creation fails

What you see: Clicking the upgrade/subscribe button does nothing, or shows an error like "Failed to create checkout session." The browser console shows a `500` error on `/api/stripe/checkout`.

Most likely causes:

1. `STRIPE_SECRET_KEY` is missing or wrong.
2. The price ID (`STRIPE_PRICE_SUBSCRIBER_MONTHLY`, etc.) does not match a real price in your Stripe account.
3. The user is not logged in (checkout requires authentication).

Fix — Step by step:

Step 1: Check that the user is logged in The checkout API requires authentication. Confirm the user is logged into the app before clicking "Subscribe."

Step 2: Verify the Stripe secret key In Vercel → **Settings** → **Environment Variables**, confirm `STRIPE_SECRET_KEY` is set. It should start with `sk_live_` (production) or `sk_test_` (test mode). Never use `pk_` (publishable) as the secret key.

Step 3: Verify the price IDs

1. Go to dashboard.stripe.com → **Products** → click a product → click a price
2. Copy the Price ID (starts with `price_...`)
3. In Vercel → **Environment Variables**, confirm `STRIPE_PRICE_SUBSCRIBER_MONTHLY` (and other price IDs) match the IDs in Stripe exactly

Step 4: Check for live/test mode mismatch If your Stripe account is in test mode, use `sk_test_...` keys and test price IDs. If in live mode, use `sk_live_...` keys and live price IDs. Never mix test keys with live price IDs.

Step 5: Check Vercel function logs for the exact error Vercel Dashboard → your project → **Logs** → filter by `/api/stripe/checkout` → look at the error message printed by `console.error("Stripe checkout error:", err)`. This will show the exact Stripe API error.

---

8. General Problems

8.1 Environment variable not being read

What you see: A feature that requires an API key or URL is broken, but you are certain the value is set. The server acts as if the variable is empty.

Common causes and fixes:

Cause 1: The variable was added to the wrong file

For **local development**, all environment variables live in:

- `frontend/.env.local` — Next.js frontend variables
- `backend/.env` — Python backend variables

The file `.env` (without `.local`) in `frontend/` is for build-time defaults and is committed to git — do not put secrets there.

Cause 2: The variable was added to Vercel but the deployment was not re-done

After adding or changing a Vercel environment variable, you must **redeploy**:

1. Vercel Dashboard → **Deployments** → latest deployment → **Redeploy**
2. Or push a new commit to trigger an automatic deployment

Cause 3: `NEXT_PUBLIC_` prefix is missing for browser-accessible variables

Variables used in client-side code (browser) **must** start with `NEXT_PUBLIC_`. Variables used only in server-side code (API routes, middleware, `getServerSideProps`) do not need the prefix. Getting this wrong means:

- Client code that reads a non-`NEXT_PUBLIC_` var gets `undefined`
- A `NEXT_PUBLIC_` var is exposed to the browser (a security risk for secrets)

Reference:

Variable	Where used	Prefix needed
`NEXT_PUBLIC_SUPABASE_URL`	Browser + server	`NEXT_PUBLIC_`
`SUPABASE_SERVICE_ROLE_KEY`	Server only	None (never expose to browser)
`STRIPE_SECRET_KEY`	Server only	None
`NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`	Browser	`NEXT_PUBLIC_`
`GOOGLE_API_KEY`	Python backend only	Stored in `backend/.env`, not frontend

Cause 4: The dev server was not restarted after editing `.env.local`

Next.js reads `.env.local` at startup only. After editing it:

1. Press `Ctrl+C` to stop the dev server
2. Run `npm run dev` again

Cause 5: A trailing space or newline in the value

When copying API keys from a dashboard, it is easy to accidentally include a trailing space. Open the `.env` file in a text editor and verify there are no spaces after the `=` value. Quotes are optional but safe:

```
GOOGLE_API_KEY=AIzaSyB...your_key_without_trailing_space
```

8.2 Works locally but broken in production

What you see: Everything works on `localhost:3000` but the same feature fails on `https://yourapp.vercel.app`.

This is almost always one of five causes:

Cause 1: Missing environment variable in Vercel The most common cause. Check every env var your feature needs in Vercel → **Settings** → **Environment Variables**. If one is missing or set to the wrong value, the feature breaks in production only.

Cause 2: Local environment uses `http://localhost` but production uses `https://` Check these variables:

- `NEXT_PUBLIC_APP_URL` → should be `https://yourapp.vercel.app` in production, not `http://localhost:3000`
- `FRONTEND_URL` in `backend/.env` → should be your production domain in production
- Stripe `success_url` and `cancel_url` derive from `origin` and `NEXT_PUBLIC_APP_URL` — ensure it is correct

Cause 3: The Python backend URL is wrong `PYTHON_BACKEND_URL` in Vercel must point to your live Railway (or other host) backend URL, not `http://localhost:8000`. Set it to something like `https://your-backend.railway.app`.

Cause 4: CORS is blocking cross-origin requests The Python backend only allows requests from `FRONTEND_URL`. In `backend/.env` (or Railway environment variables), set:

```
FRONTEND_URL=https://yourapp.vercel.app
```

If this is wrong, browser API calls to the backend will be blocked with a CORS error.

Cause 5: Stripe is in test mode locally but needs live mode in production Confirm production uses `sk_live_...` / `pk_live_...` keys and live price IDs in Stripe.

Diagnostic checklist for "works locally, broken in production":

1. Open Vercel → **Logs** for the failing function → read the exact error
2. Compare environment variables: local `.env.local` vs. Vercel env vars — list them side by side
3. Check if the Railway backend is running (visit your Railway backend URL in the browser)
4. Open browser DevTools → **Network** tab → reproduce the failure → find the red request → read the response body

---

Quick Reference

Starting the app (local development)

Terminal A — Python AI Backend:

```
cd /Users/baba/Vermont-Health-Platform/backend
uvicorn main:app --reload --port 8000
```

Terminal B — Next.js Frontend:

```
cd /Users/baba/Vermont-Health-Platform/frontend
npm run dev
```

App is at `http://localhost:3000`. Backend health check: `http://localhost:8000/health`.

Key dashboard URLs

Service	URL
Supabase	supabase.com/dashboard
Vercel	vercel.com
Sanity Studio (local)	`http://localhost:3000/studio`
Sanity manage	sanity.io/manage
Stripe	dashboard.stripe.com
Google Cloud Console	console.cloud.google.com
Railway	railway.app

Environment variable checklist

`frontend/.env.local` (local dev):

```
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=
NEXT_PUBLIC_SANITY_PROJECT_ID=
NEXT_PUBLIC_SANITY_DATASET=production
NEXT_PUBLIC_SANITY_API_VERSION=2023-10-01
SANITY_API_TOKEN=
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=
STRIPE_PRICE_SUBSCRIBER_MONTHLY=
STRIPE_PRICE_SUBSCRIBER_YEARLY=
PYTHON_BACKEND_URL=http://localhost:8000
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

`backend/.env`:

```
GOOGLE_API_KEY=
SANITY_PROJECT_ID=
SANITY_DATASET=production
SANITY_API_TOKEN=
SUPABASE_URL=
SUPABASE_SERVICE_ROLE_KEY=
SUPABASE_JWT_SECRET=
SUPABASE_DB_URL=
FRONTEND_URL=http://localhost:3000
```

Protected routes and required roles

Route prefix	Required role
`/admin`	admin
`/advisory-hub`	advisory (or higher)
`/dashboard`	subscriber (or higher)
`/chat`	subscriber (or higher)
`/hti-dashboard`	subscriber (or higher)
`/account`	any authenticated user

Route prefix	Required role
`/onboarding`	any authenticated user

Role hierarchy (lowest to highest): `free → subscriber → student → professional → advisory → admin`

A user with `advisory` can access all routes requiring `subscriber` or lower.